AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Fall 2024-2025

Section: [*S*]

Group No: 01

Project Report On

Project Name ***[PC BUILDER BANGLADESH]***

Supervised By

Tamanna Zaman Bristy

Submitted By:

Name	ID
1. Shahadat Hossain	**23-51341-1**
2. Hamidul Islam	**23-50447-1**
3. Razia Fahmida Haque	**23-50740-1**

Obtained Marks for CO2 and CO3 (Description given in the following page)

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)

Evaluation Criteria (CO2)	Total =	Evaluation Criteria (CO3)	Total =
Requirement fulfillment		Organization of the application	
Validation		Representation and Integration of Database	
Verification		Graphical User Interface	

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Requirement fulfillment	Fails to demonstrate any understanding of real-life scenario-based project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately.	Demonstrates limited understanding of real-life scenario-based project development and functional requirement identification. The project depicted lacks coherence or relevance to real-life scenarios, and functional requirements are inaccurately identified or insufficiently described.	Presents a basic depiction of a real-life scenario-based project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details.	Effectively demonstrates a realistic scenario-based project and accurately identifies most functional requirements. The project is well-developed with appropriate complexity, and functional requirements are clearly articulated with relevant details.	Exhibits an exceptional understanding of real-life scenario-based project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system.
Verification	Fails to demonstrate any attempt to verify the system data or functional requirements. There is no evidence of understanding or implementation	Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts are incomplete or	Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may be inconsistent or	Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and thorough, with attention to	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive

	of verification processes, and data flow is not considered.	inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality.	lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow.	ensuring data integrity and appropriate data flow within the system.	and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.

CO3: Prepare and Explain a real life desktop based application synthesizing several component of C# along with development tools to adhere the given requirements.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Organization of the application	Fails to identify any suitable real time application or requirements for project development activities related to OOP.	Limited understanding about the project scopes and scenarios or identification of functional requirements.	Lacks depth or relevance to OOP project development activities and may contain inaccuracies. Real-life scenarios are mentioned, but the discussion lacks depth or clarity.	Consider and integrate the idea of several core aspects of the project along with relevance to real-life scenarios. Demonstrating a solid understanding of the application presentation.	Generalize and exhibits an exceptional understanding of project preparation according to a to real-life scenarios. Also contains proper and insightful identification of the system which is comprehensive and precise.
Representation and Integration of Database	Fails to identify and present any understanding or implementation of database. Also failed to integrate the data with the project itself.	Limited understanding of the database concepts or their proper way of using in a real time project. While some attempt may be made to implement but it is incomplete or poorly executed, leading to inadequate design.	Lacks depth or relevance to database integration with the application. Shows a basic understanding but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistency. May lack proper normalization.	Integrate the database with the forms properly and implements it with proper validation which is mostly accurate and comprehensive, ensuring the proper handling of data input and verification along with general normalization.	Exhibits an exceptional understanding and implementation of database ensuring attention to detail, and robust data manipulation procedures and contributing to the overall clarity.
Graphical User Interface	Fails to present or prepare GUI based application interfaces. There is no evidence of creating or integrating such things according to their usefulness.	Limited understanding of graphical user interfaces. Lack of design knowledge. Very poor attempt to make such things which are currently obsolete or can't be identified as coherent.	Shows a basic understanding of creating user interfaces. Most of them are interconnected but maybe some of them lack it. However, most of it can be described as user friendly.	Effectively identifies and meet the consider the simplicity. Design related works are mostly accurate and taken proper attention to ensuring a user-friendly coherent system.	Exhibits an exceptional work design following a high standard of simple and elegant work. Several controls and mechanism has been organized in a preferred way according to the coherent usage .

Table of Contents: **Page no.**

1. Chapter: 01 (Introduction)---------- 03
2. Chapter: 02 (User Story)---------- 03
3. Chapter: 03a (ER Diagram)---------- 05
4. Chapter: 03b (SQL Queries)---------- 06
5. Chapter: 04 (Screenshots)---------- 09

Chapter: 01

Introduction:

PC Builder Bangladesh is a comprehensive management system developed using Windows Form Application (WFA) in Visual Studio 2022 IDE using **C# .NET 8 framework**, following **Object-Oriented Programming (OOP) principles**. The system streamlines the management of **PC components**, **user roles**, and **order processing**, catering to three primary user roles: **Admin, Staff, and Customers**.

- **Admin** oversees system-wide operations, including user management and system maintenance.
- **Staff** handle inventory control, product management, and order processing.
- **Customers** interact with the system to customize and purchase PC components.

This report provides a structured breakdown of the functionalities available to each user role, highlighting their responsibilities and interactions within the system.

Chapter: 02

Admin Functionalities

The Admin is the owner of the system and holds the highest level of control over all features and operations. With unrestricted access, the Admin oversees user management and system maintenance. The core functionalities available to the Admin are as follows:

1. Login Functionality

The Admin is appointed internally, and therefore, a new Admin can only be added by modifying the system database directly. Unlike other user roles, the Admin does not have a separate sign-up option.

- The Admin credentials consist of a **Username** and **Password**, where the Username serves as the **Primary Key** and must be unique.
- To log in, the Admin must enter the registered Username and corresponding Password.

2. Adding a New User

The Admin has the authority to create new **Customer** and **Staff** accounts within the system.

- The Admin registers a user by entering essential details such as **Name, Contact Number, Password, Email Address, Residential Address,** and **User Role** (either Customer or Staff).
- Upon submission, the system stores the information in the database, enabling the new user to log in directly without needing to sign up separately.

3. Removing an Existing User

The Admin can remove both **Customers** and **Staff** from the system using the following methods:

- Searching for a user by their Name.
- Viewing the full list of registered users by clicking the **Search** button, which displays all Customers and Staff members.
- Selecting a user from the list and clicking the **Delete** button to permanently remove them from the system.

4. Updating User Information

The Admin has the capability to update the details of existing users.

- A **grid view** displays all user information, simplifying the selection process.
- The Admin can modify the **Contact Number, Password, Email Address,** and **Residential Address** of a Customer or Staff member.
- The **Name (Username)** is changeable, but it has to be unique as it serves as the **Primary Key** for user identification.
- Once the updated details are entered, clicking the **Update** button applies the changes and updates the database accordingly.

5. Viewing User Information

The Admin can access and monitor user details to maintain system oversight.

- Searching by **Name** allows quick access to a specific user's information.
- Clicking the **Search** button retrieves a comprehensive list of all registered users, displaying their details for review.

Staff Functionalities

The Staff members manage **inventory, stock, and order processing** within the system. Their key functionalities include:

1. Login Functionality

- Existing Staff can log in using their **Name** and **Password**.
- New Staff members must sign up by clicking the **Sign Up** button, which opens the sign up form.
- They must provide their **Name (Unique), Contact Number, Password, Email, Address,** and select **Staff** as their role.
- Upon completing the registration and clicking the **Sign Up** button, their information is added to the Staff database, allowing them to log in.

2. Adding a Product

- Staff can add new products to the inventory by entering details such as **Name, ID, Brand, Unit Price, Model, Available Quantity,** and **Storage Capacity (for RAM or ROM)**.
- Each product **ID** must be unique, as it serves as the **Primary Key**.
- The product must be assigned to a category, choosing from **Monitor, Mouse, Keyboard, CPU, Processor, Motherboard, RAM,** or **ROM**.
- Clicking the **Add Product** button adds the product to the inventory and updates the respective product category database.

3. Deleting a Product

- Staff can remove products from the inventory by selecting the appropriate **Product Category** (Monitor, Keyboard, Mouse, CPU, Processor, RAM, ROM, or Motherboard).
- Once a category is chosen, the system displays all products within that category, sorted in ascending order by **ID**.
- The desired product can be selected and deleted by clicking the **Delete** button, permanently removing it from the inventory and database.

4. Viewing Product Details

- Staff can monitor available and out-of-stock products to manage inventory effectively.
- Two grids display:
    - **Available Products**: Clicking the **Show Available Products** button retrieves all in-stock items.
    - **Stock Out Products**: Clicking the **Show Stock Out Products** button lists all products with zero stock.
- This feature helps Staff track inventory levels and manage stock efficiently.

5. Viewing Orders

- Staff can access the pending **Customer Orders** through the **Show Order** grid.
- Clicking the **Placed Order** button displays a list of pending orders, sorted by **Serial Number**.
- Clicking the **Placed PC Order** button displays a list of pending customized PC orders, sorted by **Serial Number**.
- The order list includes details such as **Serial Number, Name (Buyer), Quantity,** and **Total Price**.
- This feature assists Staff in processing and fulfilling orders efficiently.

By leveraging these functionalities, the Staff ensures smooth inventory management and efficient order handling within the system.

Customer Functionalities

Customers interact with the system primarily to browse and purchase PC components. Their key functionalities include:

1. Login Functionality

- Existing Customers can log in using their **Name** and **Password**.
- New Customers must sign up by clicking the **Sign Up** button, filling in their **Name (Unique), Contact Number, Password, Email, Address,** and selecting **Customer** as their role.
- Upon registration, their details are added to the Customer database, allowing them to log in and place orders.

2. Ordering a Customized PC

- Customers can build and order a custom PC by selecting individual components.
- Clicking **Order PC** opens a selection dashboard where they choose parts from categories like **Monitor, Mouse, Keyboard, Processor, CPU, Motherboard, RAM,** and **ROM**.
- The system displays all available products.
- Customers can add products to the cart, select multiple products, and view the total price dynamically.
- Before placing an order, the customer can enter a **Billing Name** (if someone else will receive the order).
- Clicking **Order** redirects them to the payment page, where **Cash on Delivery (COD)** is currently the only available option.

3. Ordering Individual Products

- Customers can also purchase individual components without building a full PC.
- They select a product from the relevant **category**, choose the **quantity**, and follow the same order process as in **Order PC**.

With these functionalities, the system provides Customers with a seamless experience in purchasing PC components while maintaining a structured inventory and order management process.

Chapter: 03

Entity Relationship Diagram:

Normalization:

Admin:

2NF:

Name (Primary Key), Pass

Staff:

2NF:

Name (Primary Key), Pass, Mail, Number, Address

Customer:

2NF:

Name (Primary Key), Pass, Mail, Number, Address

PC Components:

2NF:

ID (Primary Key), Name, Model, Price, Brand, Quantity, Storage

PC Order:

2NF:

Serial (Primary Key), Name, Products, Total_Price

Order:

2NF:

Serial (Primary Key), Buyer_Name, Quantity, Total_Price , Products

Chapter: 03
SQL SERVER Queries:

Table Creation Query For Admin Data Table:

```
CREATE TABLE [dbo].[Admin] (
    [Pass] INT        NOT NULL,
    [Name] NCHAR (10) NOT NULL,
    PRIMARY KEY CLUSTERED ([Name] ASC)
);
```

Table Creation Query For Staff Data Table:

```
CREATE TABLE [dbo].[Staff] (
   [NAME]    NCHAR (10)    NOT NULL,
   [PASS]    INT           NOT NULL,
   [NUMBER]  NCHAR (10)    NULL,
   [MAIL]    NVARCHAR (50) NULL,
   [ADDRESS] NCHAR (10)    NULL,
   PRIMARY KEY CLUSTERED ([NAME] ASC)
);
```

Table Creation Query For Customer Data Table:

```
CREATE TABLE [dbo].[Customer] (
   [NAME]    NCHAR (10)    NOT NULL,
   [PASS]    INT           NOT NULL,
   [NUMBER]  NCHAR (10)    NULL,
   [MAIL]    NVARCHAR (50) NULL,
   [ADDRESS] NCHAR (10)    NULL,
   PRIMARY KEY CLUSTERED ([NAME] ASC)
);
```

Table Creation Query For PC Order Table:

```
CREATE TABLE [dbo].[PCorder] (
   [Serial]     INT         IDENTITY (1, 1) NOT NULL,
   [Name]       VARCHAR (50) NULL,
   [Products]   VARCHAR (50) NULL,
   [TotalPrice] VARCHAR (50) NULL,
   PRIMARY KEY CLUSTERED ([Serial] ASC)
);
```

Table Creation Query For Order Table:

```
CREATE TABLE [dbo].[Order] (
   [Serial]      INT         IDENTITY (1, 1) NOT NULL,
   [Buyer_Name]  VARCHAR (50) NULL,
   [Quantity]    VARCHAR (50) NULL,
   [Total_Price] VARCHAR (50) NULL,
   PRIMARY KEY CLUSTERED ([Serial] ASC)
);
```

Table Creation Query For each PC Component Table:

CPU:

```
CREATE TABLE [dbo].[CPU] (
   [Id]      INT           NOT NULL,
   [Name]    VARCHAR (50)   NULL,
   [Price]   VARCHAR (50)   NULL,
   [Brand]   VARCHAR (50)   NULL,
```

```
    [Model]    VARCHAR (50)   NULL,
    [Quantity] VARCHAR (50)   NULL,
    [Image]    NVARCHAR (255) NULL,
    [Storage ] VARCHAR (50)   NULL,
    PRIMARY KEY CLUSTERED ([Id] ASC)
);
```

Keyboard:

```
CREATE TABLE [dbo].[Keyboard] (
    [Id]       INT           NOT NULL,
    [Name]     VARCHAR (50)   NULL,
    [Price]    VARCHAR (50)   NULL,
    [Brand]    VARCHAR (50)   NULL,
    [Model]    VARCHAR (50)   NULL,
    [Quantity] VARCHAR (50)   NULL,
    [Image]    NVARCHAR (255) NULL,
    [Storage ] VARCHAR (50)   NULL,
    PRIMARY KEY CLUSTERED ([Id] ASC)
);
```

Monitor:

```
CREATE TABLE [dbo].[Monitor] (
    [Id]       INT           NOT NULL,
    [Name]     VARCHAR (50)   NULL,
    [Price]    VARCHAR (50)   NULL,
    [Brand]    VARCHAR (50)   NULL,
    [Model]    VARCHAR (50)   NULL,
    [Quantity] VARCHAR (50)   NULL,
    [Image]    NVARCHAR (255) NULL,
    [Storage ] VARCHAR (50)   NULL,
    PRIMARY KEY CLUSTERED ([Id] ASC)
);
```

Motherboard:

```
CREATE TABLE [dbo].[Motherboard] (
    [Id]       INT           NOT NULL,
    [Name]     VARCHAR (50)   NULL,
    [Price]    VARCHAR (50)   NULL,
    [Brand]    VARCHAR (50)   NULL,
    [Model]    VARCHAR (50)   NULL,
    [Quantity] VARCHAR (50)   NULL,
    [Image]    NVARCHAR (255) NULL,
    [Storage ] VARCHAR (50)   NULL,
```

```
   PRIMARY KEY CLUSTERED ([Id] ASC)
);
```

Mouse:

```
CREATE TABLE [dbo].[Mouse] (
   [Id]      INT           NOT NULL,
   [Name]     VARCHAR (50)   NULL,
   [Price]    VARCHAR (50)   NULL,
   [Brand]    VARCHAR (50)   NULL,
   [Model]    VARCHAR (50)   NULL,
   [Quantity] VARCHAR (50)   NULL,
   [Image]    NVARCHAR (255) NULL,
   [Storage ] VARCHAR (50)   NULL,
   PRIMARY KEY CLUSTERED ([Id] ASC)
);
```

Processor:

```
CREATE TABLE [dbo].[Processor] (
   [Id]      INT           NOT NULL,
   [Name]     VARCHAR (50)   NULL,
   [Price]    VARCHAR (50)   NULL,
   [Brand]    VARCHAR (50)   NULL,
   [Model]    VARCHAR (50)   NULL,
   [Quantity] VARCHAR (50)   NULL,
   [Image]    NVARCHAR (255) NULL,
   [Storage ] VARCHAR (50)   NULL,
   PRIMARY KEY CLUSTERED ([Id] ASC)
);
```

Ram:

```
CREATE TABLE [dbo].[Ram] (

   [Id]      INT           NOT NULL,
   [Name]     VARCHAR (50)   NULL,
   [Price]    VARCHAR (50)   NULL,
   [Brand]    VARCHAR (50)   NULL,
   [Model]    VARCHAR (50)   NULL,
   [Quantity] VARCHAR (50)   NULL,
   [Image]    NVARCHAR (255) NULL,
   [Storage ] VARCHAR (50)   NULL,
   PRIMARY KEY CLUSTERED ([Id] ASC)
);
```

Rom:

```
CREATE TABLE [dbo].[Rom] (
    [Id]       INT           NOT NULL,
    [Name]     VARCHAR (50)   NULL,
    [Price]    VARCHAR (50)   NULL,
    [Brand]    VARCHAR (50)   NULL,
    [Model]    VARCHAR (50)   NULL,
    [Quantity] VARCHAR (50)   NULL,
    [Image]    NVARCHAR (255) NULL,
    [Storage ] VARCHAR (50)   NULL,
    PRIMARY KEY CLUSTERED ([Id] ASC)
);
```

Chapter: 04

Screenshots of the Project:

1

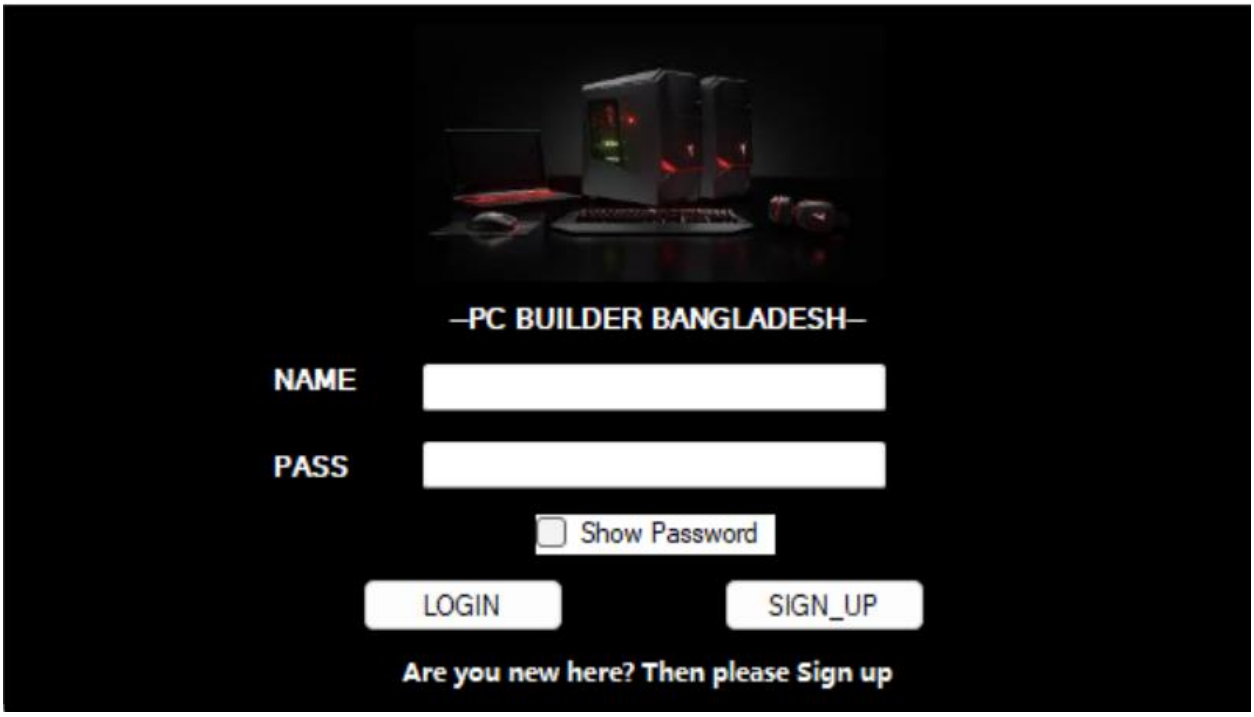


Fig: Login Form

2.



Fig: Sign Up Form for Customer & Staff

3.

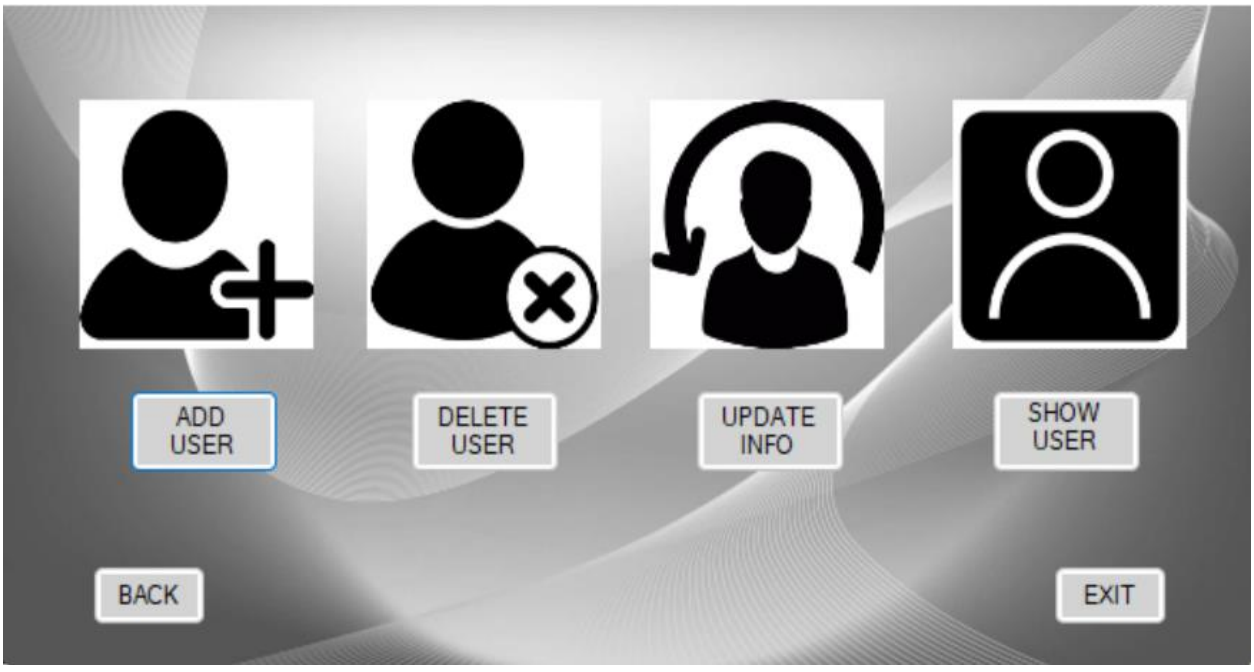


Fig: Admin Dashboard

4.

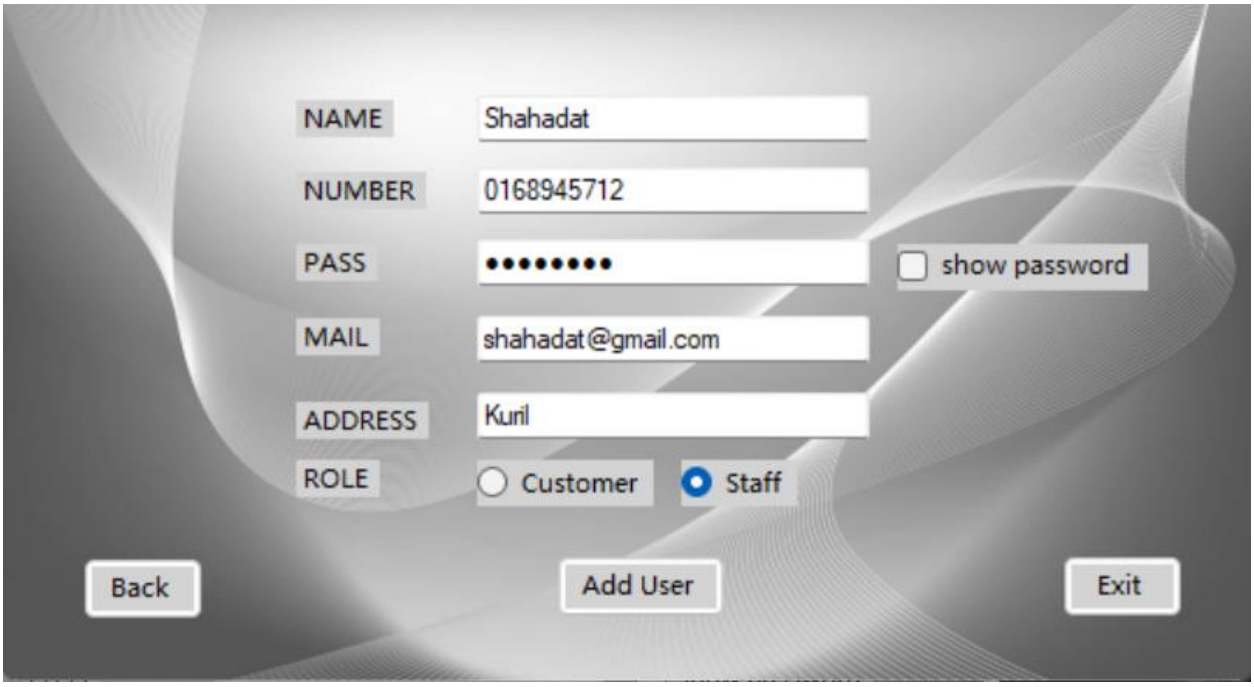


Fig: Add User Form for Admin

5.

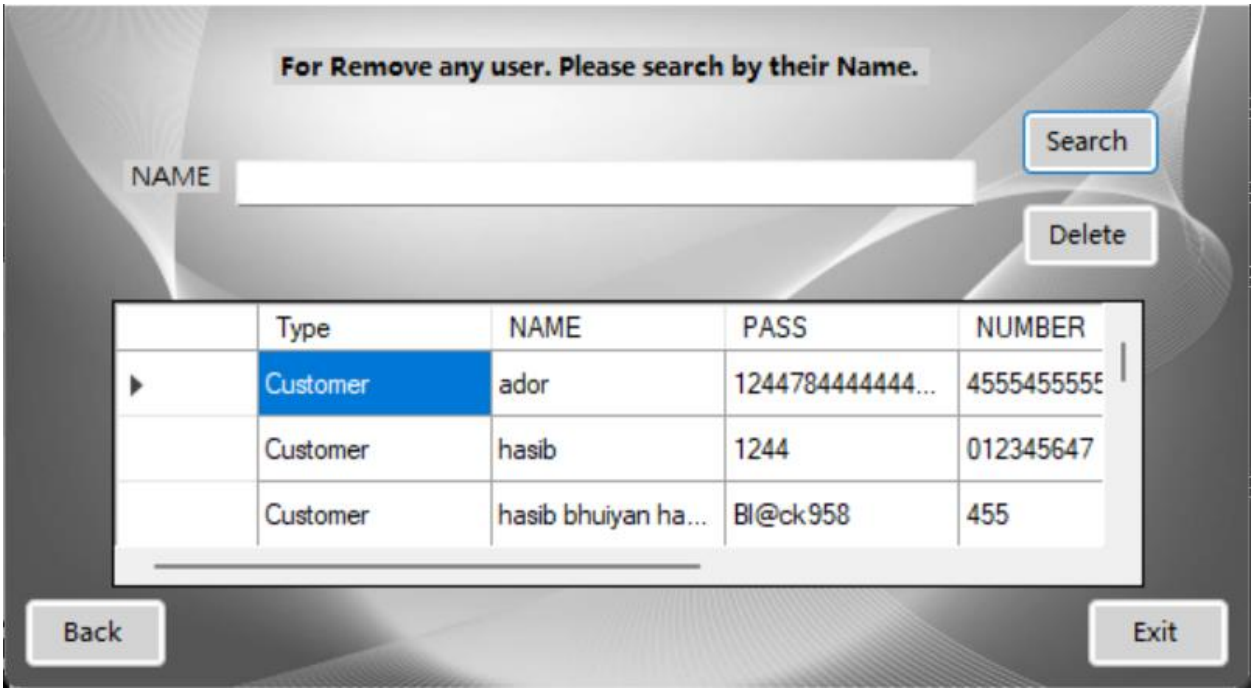


Fig: Delete User Form for Admin

6.

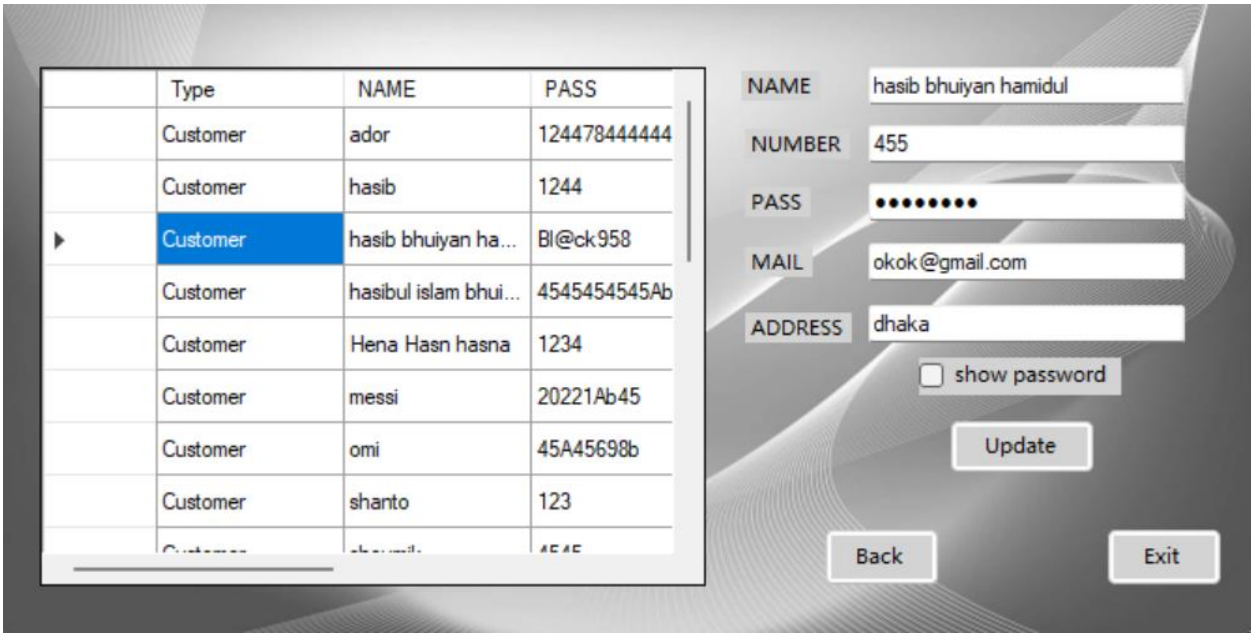


Fig: Update User Form for Admin

6.

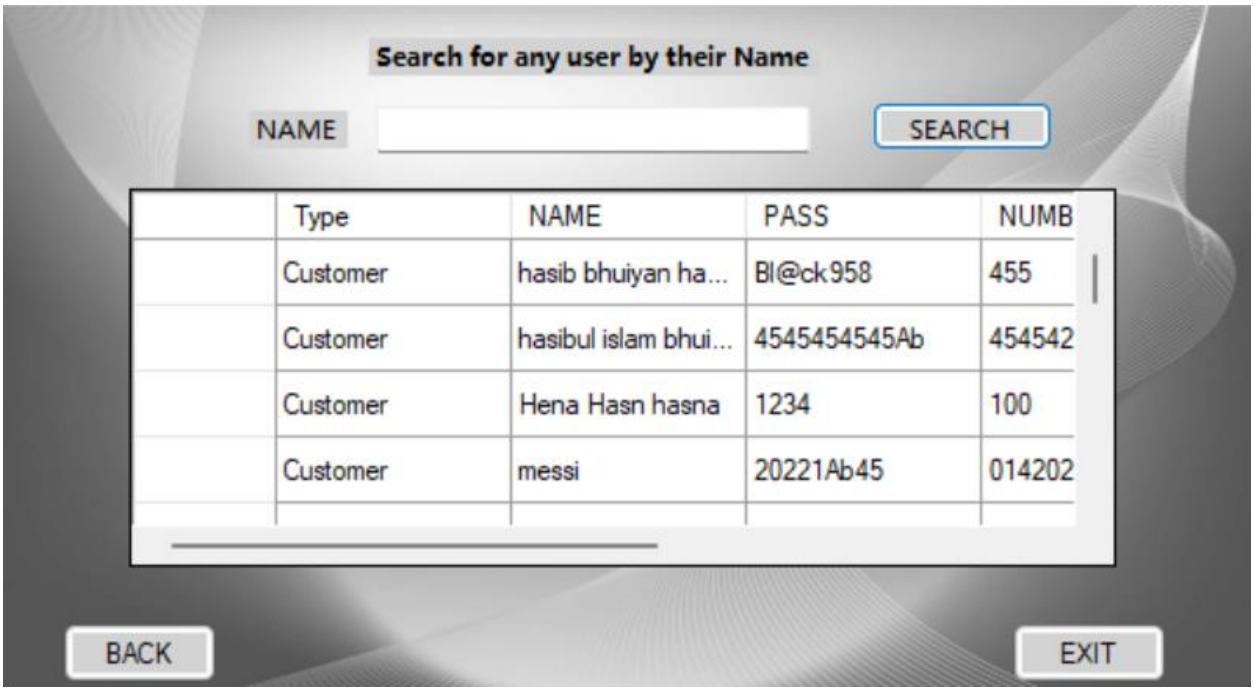


Fig: Show User Form for Admin

7.



Fig: Staff Dashboard

8.

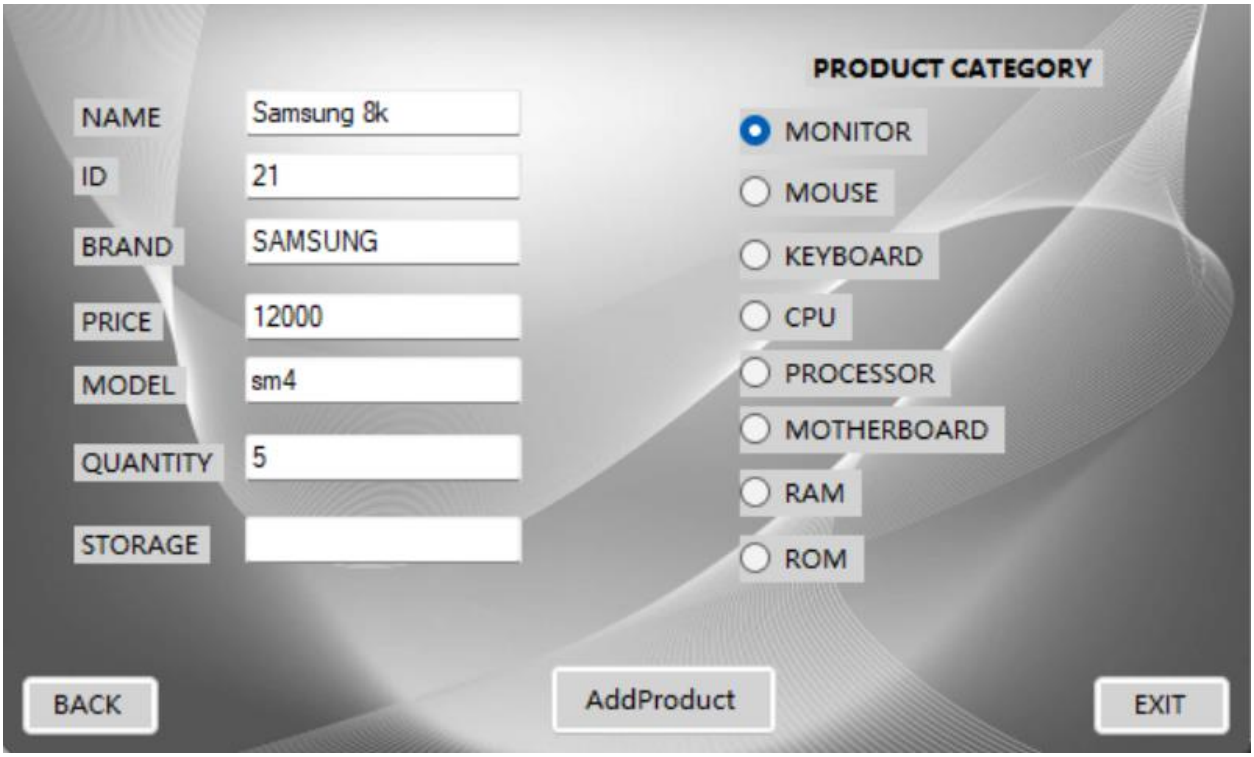


Fig: Add Product Form for Staff

9.

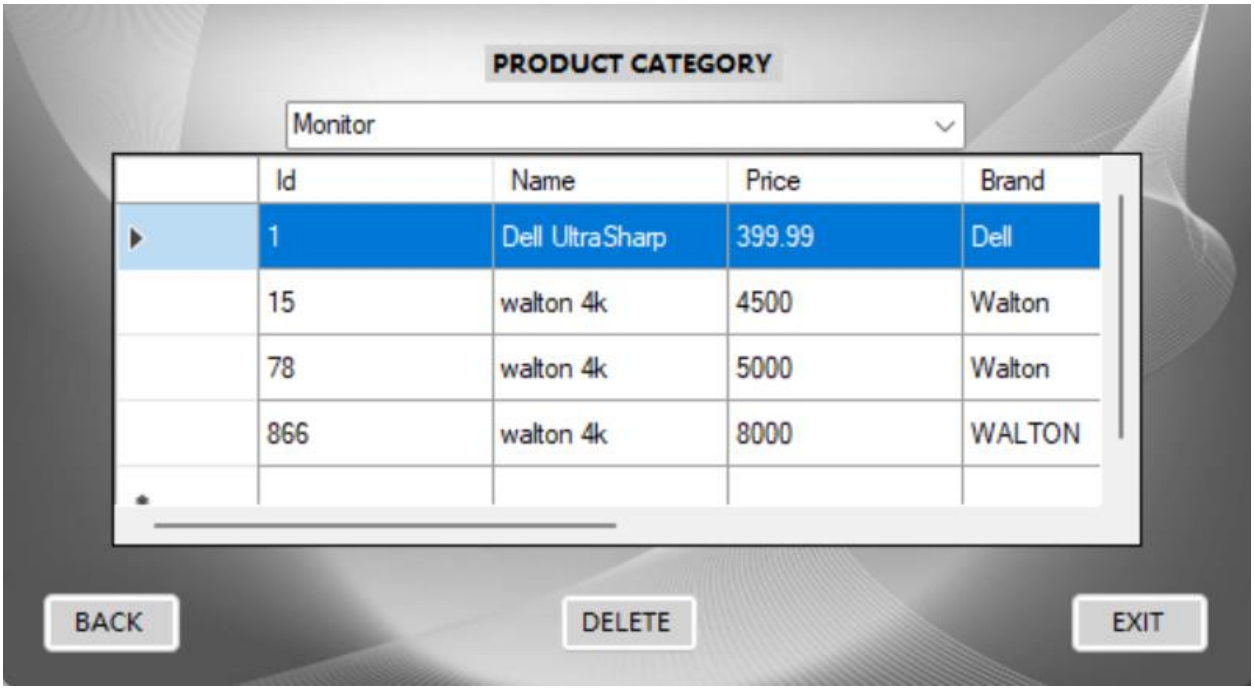


Fig: Delete Product Form for Staff

10.

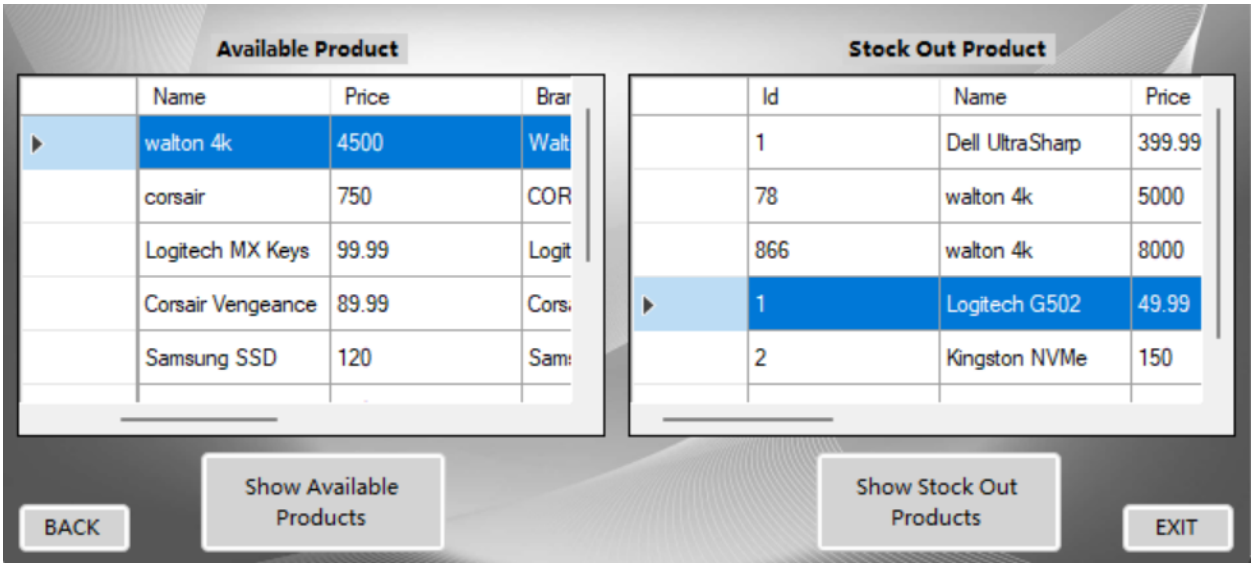


Fig: Product Details Form for Staff

11.

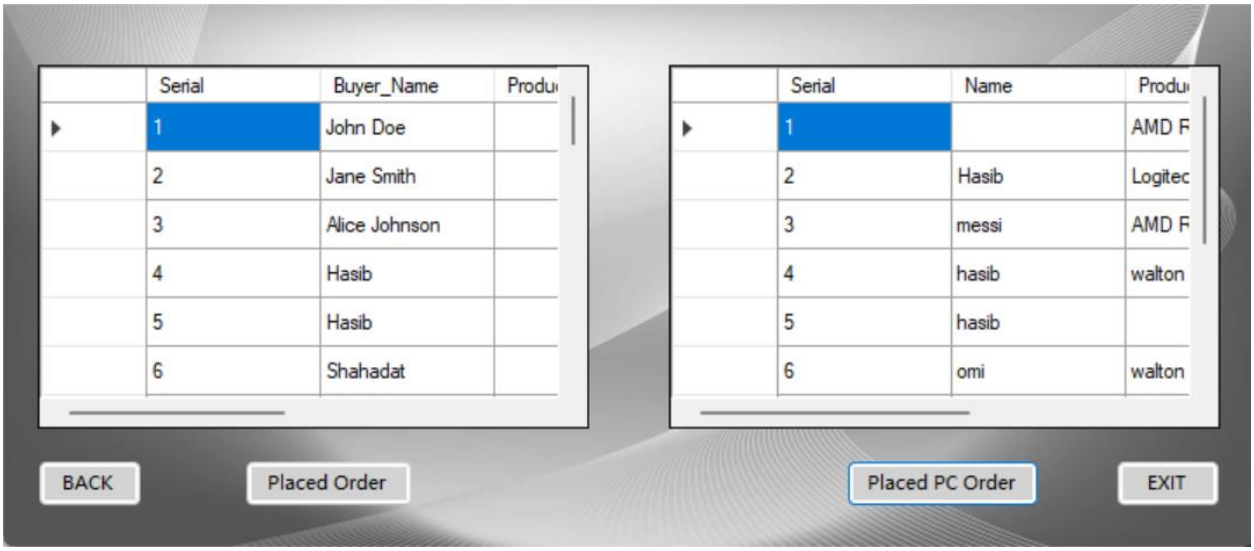


Fig: Show Order Form for Staff

12.



Fig: Customer Dashboard

13.

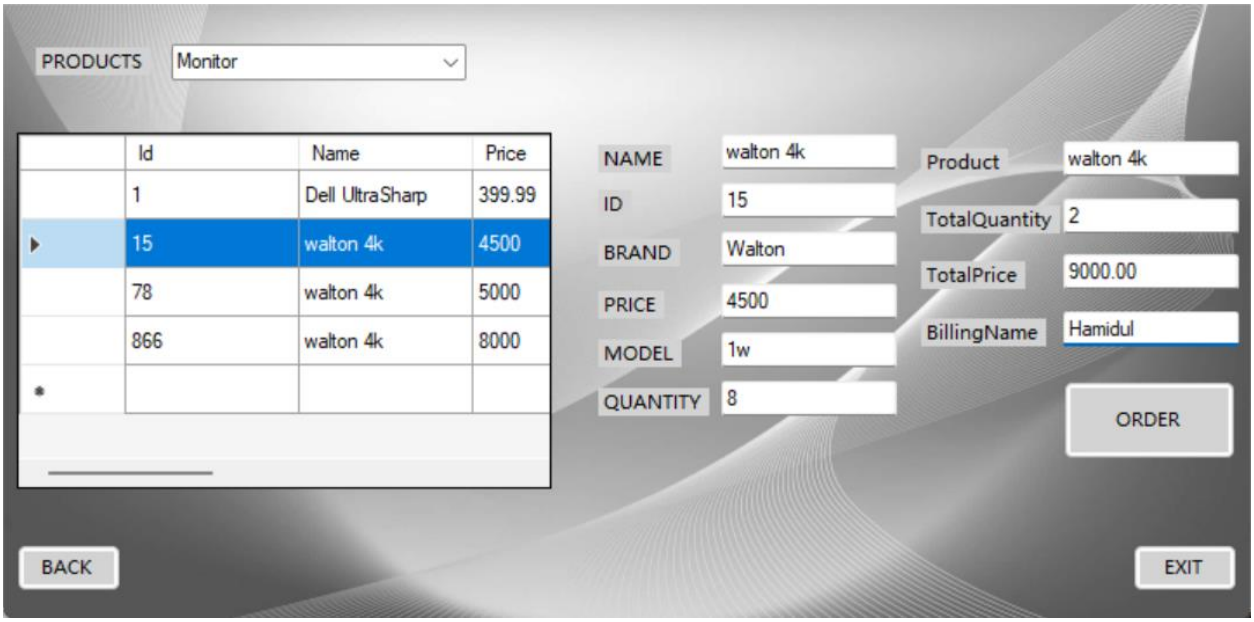


Fig: Product Order Form for Customer

14.

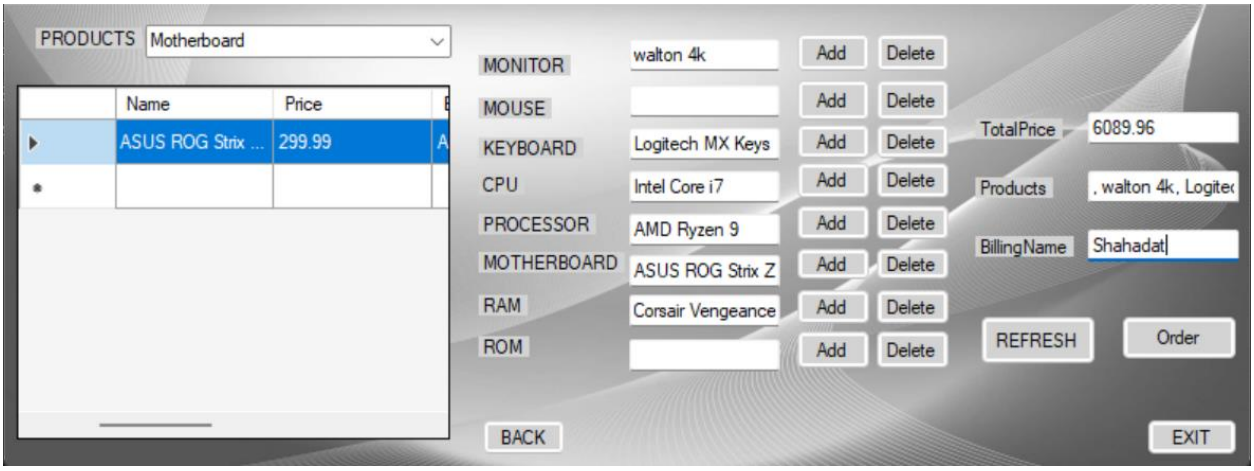


Fig: PC Order Form for Customer

15.

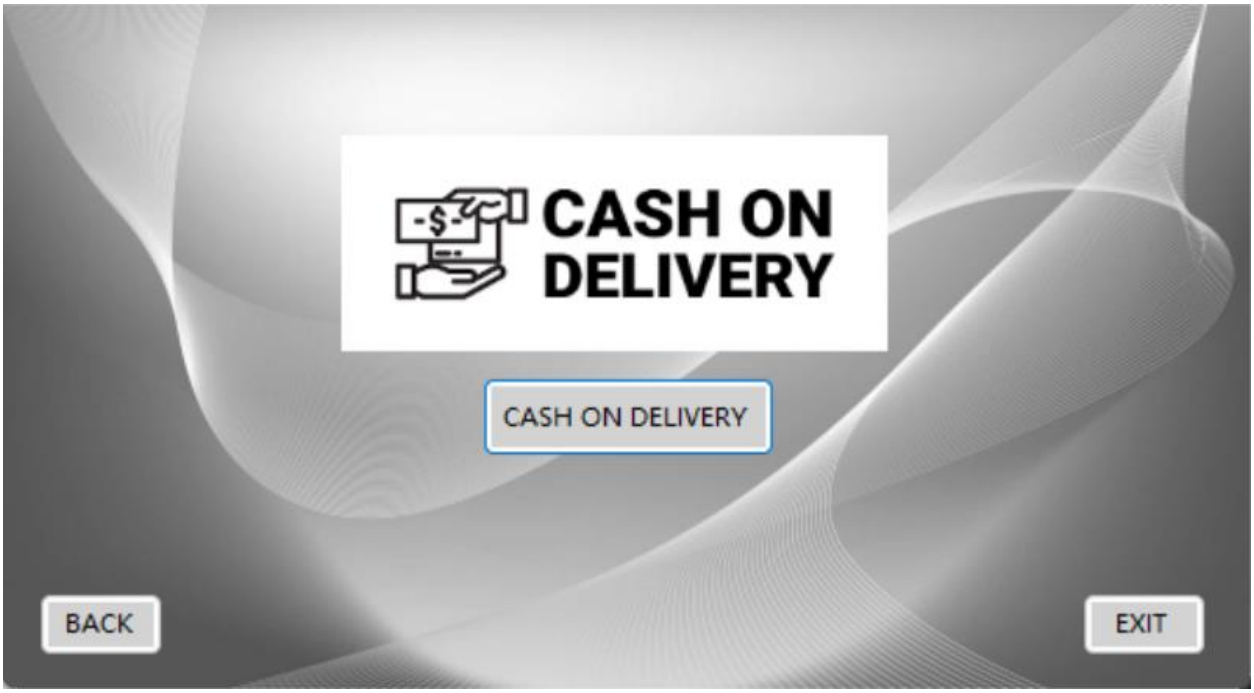
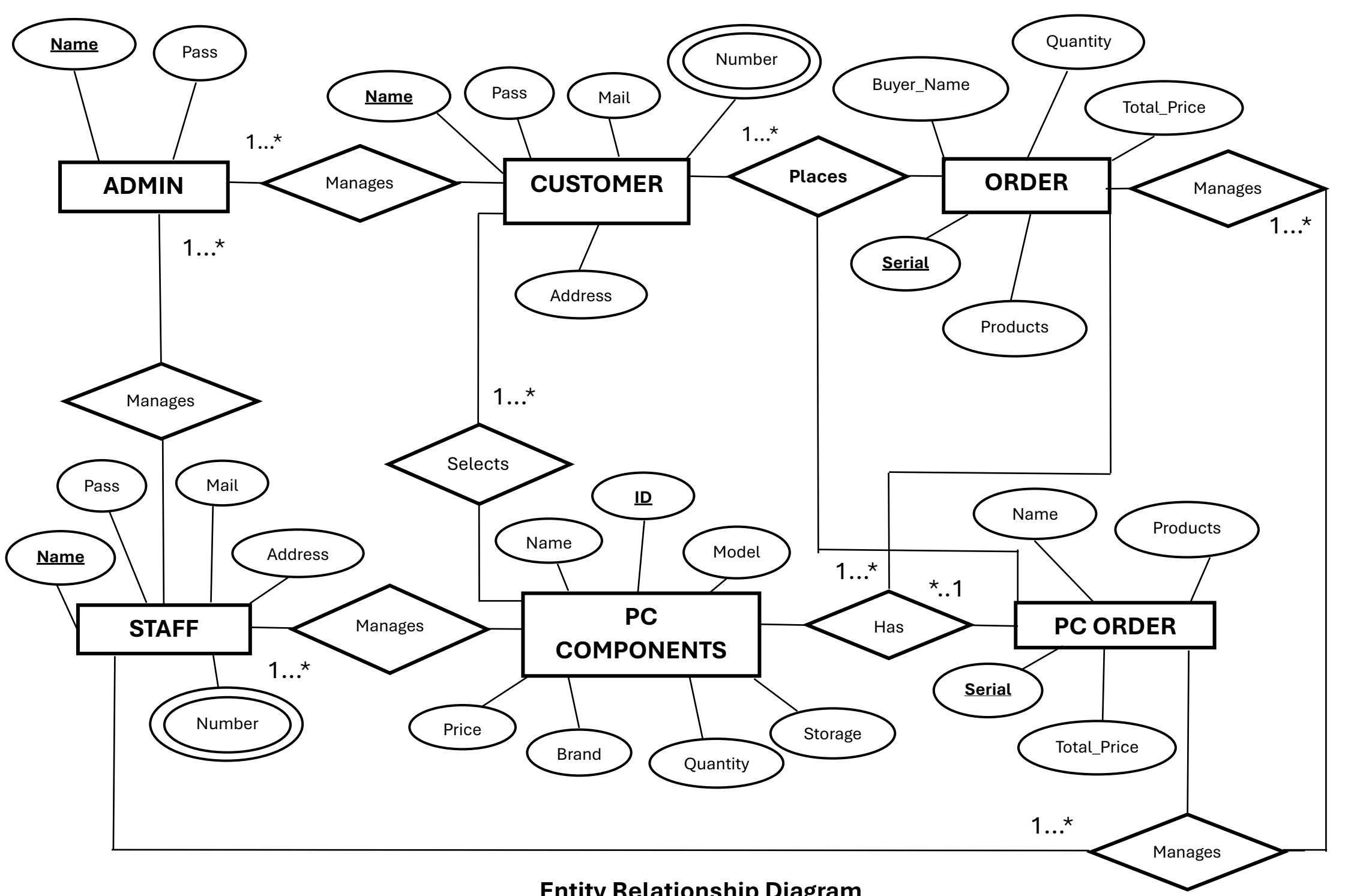


Fig: Payment Form for Customer



Entity Relationship Diagram